



ConvNet *architecture patterns*

This chapter covers

- The modularity-hierarchy-reuse formula for model architecture
- An overview of standard best practices for building ConvNets: residual connections, batch normalization, and depthwise separable convolutions
- Ongoing design trends for computer vision models

A model’s “architecture” is the sum of the choices that went into creating it: which layers to use, how to configure them, in what arrangement to connect them. These choices define the *hypothesis space* of your model: the space of possible functions that gradient descent can search over, parameterized by the model’s weights. Like feature engineering, a good hypothesis space encodes *prior knowledge* that you have about the problem at hand and its solution. For instance, using convolution layers means that you know in advance that the relevant patterns present in your input

images are translation-invariant. To effectively learn from data, you need to make assumptions about what you’re looking for.

Model architecture is often the difference between success and failure. If you make inappropriate architecture choices, your model may be stuck with suboptimal metrics, and no amount of training data will save it. Inversely, a good model architecture will accelerate learning and will enable your model to make efficient use of the training data available, reducing the need for large datasets. A good model architecture is one that *reduces the size of the search space* or otherwise *makes it easier to converge to a good point of the search space*. Just like feature engineering and data curation, model architecture is all about *making the problem simpler* for gradient descent to solve—and remember that gradient descent is a pretty stupid search process, so it needs all the help it can get.

Model architecture is more an art than a science. Experienced machine learning engineers are able to intuitively cobble together high-performing models on their first try, while beginners often struggle to create a model that trains at all. The keyword here is *intuitively*: no one can give you a clear explanation of what works and what doesn’t. Experts rely on pattern matching, an ability that they acquire through extensive practical experience. You’ll develop your own intuition throughout this book. However, it’s not *all* about intuition either—there isn’t much in the way of actual science, but like in any engineering discipline, there are best practices.

In the following sections, we’ll review a few essential ConvNet architecture best practices, in particular, *residual connections*, *batch normalization*, and *separable convolution*. Once you master how to use them, you will be able to build highly effective image models. We will demonstrate how to apply them on our dogs-versus-cats classification problem.

Let’s start from the bird’s-eye view: the modularity-hierarchy-reuse (MHR) formula for system architecture.

9.1 Modularity, hierarchy, and reuse

If you want to make a complex system simpler, there’s a universal recipe you can apply: just structure your amorphous soup of complexity into *modules*, organize the modules into a *hierarchy*, and start *reusing* the same modules in multiple places as appropriate (“reuse” is another word for *abstraction*). That’s the modularity-hierarchy-reuse (MHR) formula (see figure 9.1), and it underlies system architecture across pretty much every domain where the term *architecture* is used. It’s at the heart of the organization of any system of meaningful complexity, whether it’s a cathedral, your own body, the US Navy, or the Keras codebase.

If you’re a software engineer, you’re already keenly familiar with these principles: an effective codebase is one that is modular, hierarchical, and where you don’t reimplement the same thing twice but instead rely on reusable classes and functions. If you factor your code by following these principles, you could say you’re doing “software architecture.”

Deep learning itself is simply the application of this recipe to continuous optimization via gradient descent: you take a classic optimization technique (gradient descent

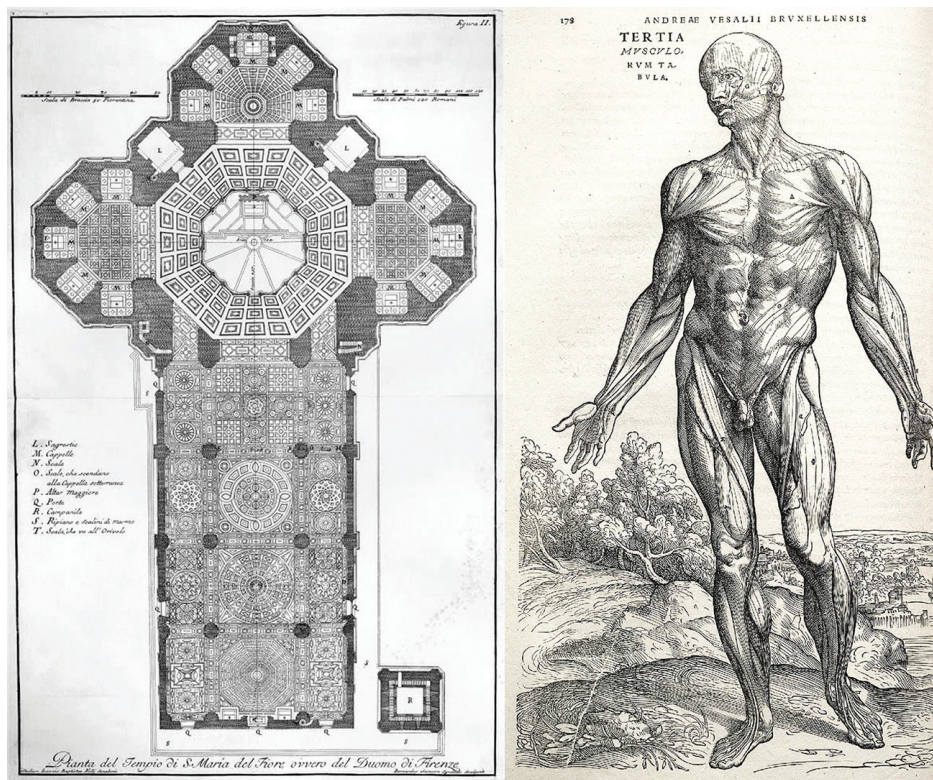


Figure 9.1 Complex systems follow a hierarchical structure and are organized into distinct modules, which are reused multiple times (such as your 4 limbs, which are all variants of the same blueprint, or your 20 fingers).

over a continuous function space), and you structure the search space into modules (layers), organized into a deep hierarchy (often just a stack, the simplest kind of hierarchy), where you reuse whatever you can (for instance, convolutions are all about reusing the same information in different spatial locations).

Likewise, deep learning model architecture is primarily about making clever use of modularity, hierarchy, and reuse. You'll notice that all popular ConvNet architectures are not only structured into layers, they're structured into repeated groups of layers (called `_blocks_` or `_modules_`). For instance, Xception architecture (used in the previous chapter) is structured into repeated `SeparableConv - SeparableConv - MaxPooling` blocks (see figure 9.2).

Further, most ConvNets often feature pyramid-like structures (*feature hierarchies*). Recall, for example, the progression in the number of convolution filters we used in the first ConvNet we built in the previous chapter: 32, 64, 128. The number of filters grows with layer depth, while the size of the feature maps shrinks accordingly. You'll notice the same pattern in the blocks of the Xception model (see figure 9.2).

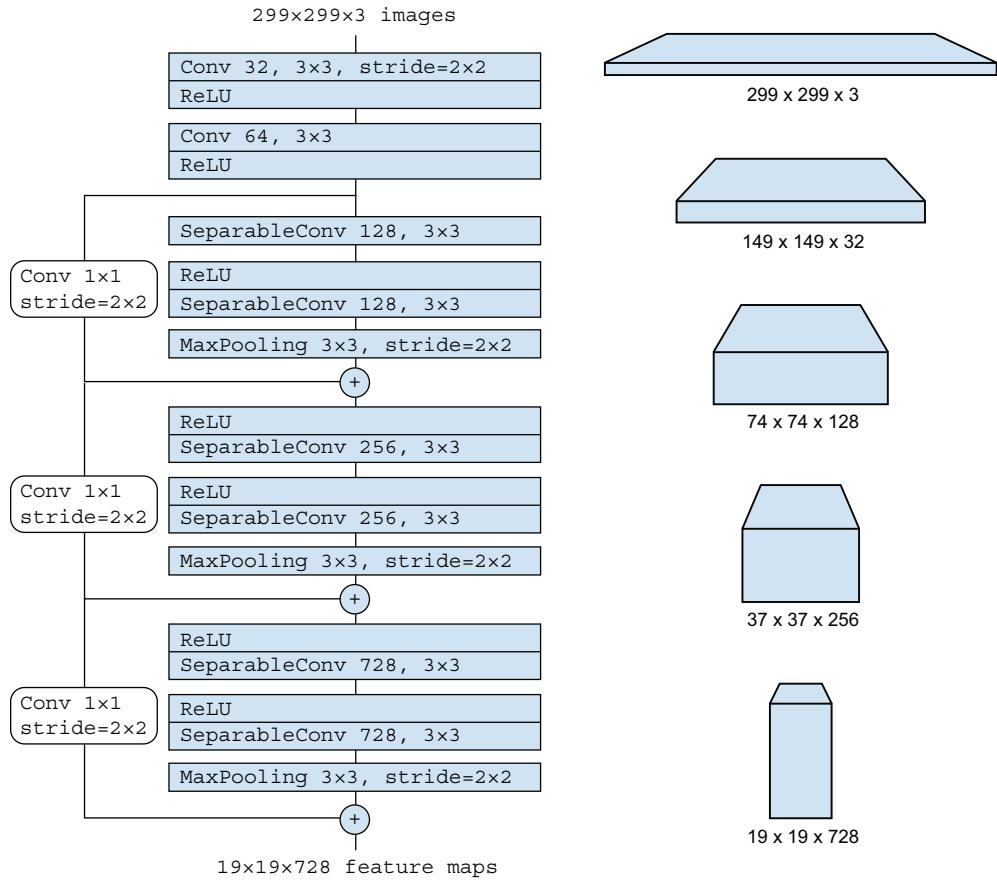


Figure 9.2 The “entry flow” of the Xception architecture: note the repeated layer blocks and the gradually shrinking and deepening feature maps, going from 299 x 299 x 3 to 19 x 19 x 728.

Deeper hierarchies are intrinsically good because they encourage feature reuse and, therefore, abstraction. In general, a deep stack of narrow layers performs better than a shallow stack of large layers. However, there’s a limit to how deep you can stack layers: the problem of *vanishing gradients*. This leads us to our first essential model architecture pattern: residual connections.

On the importance of ablation studies in deep learning research

Deep learning architectures are often more *evolved* than designed—they were developed by repeatedly trying things and selecting what seemed to work. Much like in biological systems, if you take any complicated experimental deep learning setup,

(continued)

chances are you can remove a few modules (or replace some trained features with random ones) with no loss of performance.

This is made worse by the incentives that deep learning researchers face: by making a system more complex than necessary, they can make it appear more interesting or more novel and thus increase their chances of getting a paper through the peer review process. If you read lots of deep learning papers, you will notice that they're often optimized for peer review in both style and content in ways that actively hurt clarity of explanation and reliability of results. For instance, mathematics in deep learning papers is rarely used for clearly formalizing concepts or deriving unobvious results—rather, it gets used as a *signal of seriousness*, like an expensive suit on a salesperson.

The goal of research shouldn't be merely to publish but to *generate reliable knowledge*. Crucially, *understanding causality* in your system is the most straightforward way to generate reliable knowledge. And there's a very low-effort way to look into causality: *ablation studies*. Ablation studies consist of systematically trying to remove parts of a system—that is, make it simpler—to identify where its performance actually comes from. If you find that $X + Y + Z$ gives you good results, also try X , Y , Z , $X + Y$, $X + Z$, $Y + Z$ and see what happens.

If you become a deep learning researcher, cut through the noise in the research process: do ablation studies for your models. Always ask, could there be a simpler explanation? Is this added complexity really necessary? Why?

9.2 *Residual connections*

You probably know about the game of *telephone*, also called *Chinese whispers* in the UK and *téléphone arabe* in France, where an initial message is whispered in the ear of a player, who then whispers it in the ear of the next player, and so on. The final message ends up bearing little resemblance to its original version. It's a fun metaphor for the cumulative errors that occur in sequential transmission over a noisy channel.

As it happens, backpropagation in a sequential deep learning model is pretty similar to the game of telephone. You've got a chain of functions, like this one:

$$y = f_4(f_3(f_2(f_1(x))))$$

The name of the game is to adjust the parameters of each function in the chain based on the error recorded on the output of f_4 (the loss of the model). To adjust f_1 , you'll need to percolate error information through f_2 , f_3 , and f_4 . However, each successive function in the chain introduces some amount of noise in the process. If your function chain is too deep, this noise starts overwhelming gradient information, and backpropagation stops working. Your model won't train at all. This is called the *vanishing gradients* problem.

The fix is simple: just force each function in the chain to be nondestructive—to retain a noiseless version of the information contained in the previous input. The easiest way to implement this is called a *residual connection*. It's dead easy: just add the input of a layer or block of layers back to its output (see figure 9.3). The residual connection acts as an *information shortcut* around destructive or noisy blocks (such as blocks that contain ReLU activations or dropout layers), enabling error gradient information from early layers to propagate noiselessly through a deep network. This technique was introduced in 2015 with the ResNet family of models (developed by He et al. at Microsoft).¹

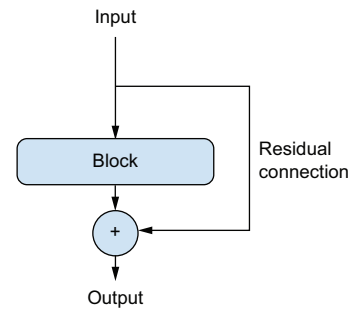


Figure 9.3 A residual connection around a processing block

In practice, you'd implement a residual connection like the following listing.

Listing 9.1 A residual connection in pseudocode

```
x = ...
residual = x
x = block(x)
x = add([x, residual])
```

Adds the original input to the layer's output. The final output will thus always preserve full information about the original input.

Some input tensor

Saves a reference to the original input. This is called the residual.

This computation block can potentially be destructive or noisy, and that's fine.

Note that adding the input back to the output of a block implies that the output should have the same shape as the input. This is not the case if your block includes convolutional layers with an increased number of filters or a max pooling layer. In such cases, use a 1×1 Conv2D layer with no activation to linearly project the residual to the desired output shape. You'd typically use `padding="same"` in the convolution layers in your target block to avoid spatial downsampling due to padding, and you'd use strides in the residual projection to match any downsampling caused by a max pooling layer.

Listing 9.2 The target block changing the number of output filters

```
import keras
from keras import layers

inputs = keras.Input(shape=(32, 32, 3))
```

¹ Kaiming He et al., “Deep Residual Learning for Image Recognition,” Conference on Computer Vision and Pattern Recognition (2015), <https://arxiv.org/abs/1512.03385>.


```

x = layers.Conv2D(32, 3, activation="relu")(inputs)
residual = x
x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
residual = layers.Conv2D(64, 1)(residual)
x = layers.add([x, residual])

```

Sets aside the residual

Now the block output and the residual have the same shape and can be added.

The residual only had 32 filters, so we use a 1 x 1 Conv2D to project it to the correct shape.

This is the layer around which we create a residual connection: it increases the number of output filters from 32 to 64. We use padding="same" to avoid downsampling due to padding.

Listing 9.3 The target block including a max pooling layer

```

inputs = keras.Input(shape=(32, 32, 3))
x = layers.Conv2D(32, 3, activation="relu")(inputs)
residual = x
x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
x = layers.MaxPooling2D(2, padding="same")(x)
residual = layers.Conv2D(64, 1, strides=2)(residual)
x = layers.add([x, residual])

```

This is the block of two layers around which we create a residual connection: it includes a 2 x 2 max pooling layer. We use padding="same" in both the convolution layer and the max pooling layer to avoid downsampling due to padding.

Sets aside the residual

We use strides=2 in the residual projection to match the downsampling created by the max pooling layer.

Now the block output and the residual have the same shape and can be added.

To make these ideas more concrete, here's an example of a simple ConvNet structured into a series of blocks, each made of two convolution layers and one optional max pooling layer, with a residual connection around each block:

```

inputs = keras.Input(shape=(32, 32, 3))
x = layers.Rescaling(1.0 / 255)(inputs)

def residual_block(x, filters, pooling=False):
    residual = x
    x = layers.Conv2D(filters, 3, activation="relu", padding="same")(x)
    x = layers.Conv2D(filters, 3, activation="relu", padding="same")(x)
    if pooling:
        x = layers.MaxPooling2D(2, padding="same")(x)
        residual = layers.Conv2D(filters, 1, strides=2)(residual)
    x = layers.add([x, residual])

```

If we use max pooling, we add a strided convolution to project the residual to the expected shape.

Utility function to apply a convolutional block with a residual connection, with an option to add max pooling

```

elif filters != residual.shape[-1]:
    residual = layers.Conv2D(filters, 1)(residual)
x = layers.add([x, residual])
return x

x = residual_block(x, filters=32, pooling=True)
x = residual_block(x, filters=64, pooling=True)
x = residual_block(x, filters=128, pooling=False)

x = layers.GlobalAveragePooling2D()(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs=inputs, outputs=outputs)

```

If we don't use max pooling, we only project the residual if the number of channels has changed.

First block

Second block. Note the increasing filter count in each block.

The last block doesn't need a max pooling layer, since we will apply global average pooling right after it.

Let's take a look at the model summary:

```

>>> model.summary()
Model: "functional"

```

Layer (type)	Output Shape	Param #	Connected to
input_layer_2 (InputLayer)	(None, 32, 32, 3)	0	-
rescaling (Rescaling)	(None, 32, 32, 3)	0	input_layer_2[0][0]
conv2d_6 (Conv2D)	(None, 32, 32, 32)	896	rescaling[0][0]
conv2d_7 (Conv2D)	(None, 32, 32, 32)	9,248	conv2d_6[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 32)	0	conv2d_7[0][0]
conv2d_8 (Conv2D)	(None, 16, 16, 32)	128	rescaling[0][0]
add_2 (Add)	(None, 16, 16, 32)	0	max_pooling2d_1[0]... conv2d_8[0][0]
conv2d_9 (Conv2D)	(None, 16, 16, 64)	18,496	add_2[0][0]
conv2d_10 (Conv2D)	(None, 16, 16, 64)	36,928	conv2d_9[0][0]
max_pooling2d_2 (MaxPooling2D)	(None, 8, 8, 64)	0	conv2d_10[0][0]

conv2d_11 (Conv2D)	(None, 8, 8, 64)	2,112	add_2[0][0]
add_3 (Add)	(None, 8, 8, 64)	0	max_pooling2d_2[0]... conv2d_11[0][0]
conv2d_12 (Conv2D)	(None, 8, 8, 128)	73,856	add_3[0][0]
conv2d_13 (Conv2D)	(None, 8, 8, 128)	147,584	conv2d_12[0][0]
conv2d_14 (Conv2D)	(None, 8, 8, 128)	8,320	add_3[0][0]
add_4 (Add)	(None, 8, 8, 128)	0	conv2d_13[0][0], conv2d_14[0][0]
global_average_pool... (GlobalAveragePooli...	(None, 128)	0	add_4[0][0]
dense (Dense)	(None, 1)	129	global_average_poo...

Total params: 297,697 (1.14 MB)
 Trainable params: 297,697 (1.14 MB)
 Non-trainable params: 0 (0.00 B)

With residual connections, you can build networks of arbitrary depth, without having to worry about vanishing gradients. Now, let's move on to the next essential ConvNet architecture pattern: *batch normalization*.

9.3 *Batch normalization*

Normalization in machine learning is a broad category of methods that seek to make different samples seen by a machine learning model more similar to each other, which helps the model learn and generalize well to new data. The most common form of data normalization is one you've seen several times in this book already: centering the data on zero by subtracting the mean from the data and giving the data a unit standard deviation by dividing the data by its standard deviation. In effect, this makes the assumption that the data follows a normal (or Gaussian) distribution and makes sure this distribution is centered and scaled to unit variance:

```
normalized_data = (data - np.mean(data, axis=...)) / np.std(data, axis=...)
```

Previous examples you saw in this book normalized data before feeding it into models. But data normalization may be a concern after every transformation performed by the network: even if the data entering a `Dense` or `Conv2D` network has a 0 mean and unit variance, there's no reason to expect a priori that this will be the case for the data coming out. Could normalizing intermediate activations help?

Batch normalization does just that. It's a type of layer (`BatchNormalization` in Keras) introduced in 2015 by Ioffe and Szegedy;² it can adaptively normalize data even as the mean and variance change over time during training. During training, it uses the mean and variance of the current batch of data to normalize samples, and during inference (when a big enough batch of representative data may not be available), it uses an exponential moving average of the batchwise mean and variance of the data seen during training.

Although Ioffe and Szegedy's original paper suggested that batch normalization operates by “reducing internal covariate shift,” no one really knows for sure why batch normalization helps. There are various hypotheses but no certitudes. You'll find that this is true of many things in deep learning—deep learning is not an exact science but a set of ever-changing, empirically derived engineering best practices, woven together by unreliable narratives. You will sometimes feel like the book you have in hand tells you *how* to do something but doesn't quite satisfactorily say *why* it works: that's because we know the how but we don't know the why. Whenever a reliable explanation is available, we make sure to mention it. Batch normalization isn't one of those cases.

In practice, the main effect of batch normalization appears to be that it helps with gradient propagation—much like residual connections—and thus allows for deeper networks. Some very deep networks can only be trained if they include multiple `BatchNormalization` layers. For instance, batch normalization is used liberally in many of the advanced ConvNet architectures that come packaged with Keras, such as ResNet50, EfficientNet, and Xception.

The `BatchNormalization` layer can be used after any layer—`Dense`, `Conv2D`, and so on:

```
x = ...
x = layers.Conv2D(32, 3, use_bias=False)(x)
x = layers.BatchNormalization()(x)
```

Because the output of the `Conv2D` layer gets normalized, the layer doesn't need its own bias vector.

NOTE Both `Dense` and `Conv2D` involve a “bias vector,” a learned variable whose purpose is to make the layer *affine* rather than purely linear. For instance, `Conv2D` returns, schematically, $y = \text{conv}(x, \text{kernel}) + \text{bias}$, and `Dense` returns $y = \text{dot}(x, \text{kernel}) + \text{bias}$. Because the normalization step will take care of centering the layer's output on zero, the bias vector is no longer needed when using `BatchNormalization`, and the layer can be created without it via the option `use_bias=False`. This makes the layer slightly leaner.

Importantly, I generally recommend placing the previous layer's activation *after* the batch normalization layer (although this is still a subject of debate). So instead of doing

² Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” *Proceedings of the 32nd International Conference on Machine Learning* (2015), <https://arxiv.org/abs/1502.03167>.

```
x = layers.Conv2D(32, 3, activation="relu")(x)
x = layers.BatchNormalization()(x)
```

you would actually do the following:

```
x = layers.Conv2D(32, 3, use_bias=False)(x)
x = layers.BatchNormalization()(x)
x = layers.Activation("relu")(x)
```

Note the lack of activation here.

We place the activation after the BatchNormalization layer.

The intuitive reason why is that batch normalization will center your inputs on zero, while your ReLU activation uses zero as a pivot for keeping or dropping activated channels: doing normalization before the activation maximizes the utilization of the ReLU. That said, this ordering best practice is not exactly critical, so if you do convolution-activation-batch normalization, your model will still train, and you won't necessarily see worse results.

NOTE Batch normalization has many quirks. One of the main ones relates to fine-tuning: when fine-tuning a model that includes `BatchNormalization` layers, I recommend leaving these layers frozen (set their `trainable` attribute to `False`). Otherwise, they will keep updating their internal mean and variance, which can interfere with the very small updates applied to the surrounding `Conv2D` layers.

Now, let's take a look at the last architecture pattern in our series: depthwise separable convolutions.

9.4 Depthwise separable convolutions

What if we told you that there's a layer you can use as a drop-in replacement for `Conv2D` that will make your model smaller (fewer trainable weight parameters), leaner (fewer floating-point operations), and cause it to perform a few percentage points better on its task? That is precisely what the *depthwise separable convolution* layer does (`SeparableConv2D` in Keras). This layer performs a spatial convolution on each channel of its input, independently, before mixing output channels via a pointwise convolution (a 1×1 convolution), as shown in figure 9.4.

This is equivalent to separating the learning of spatial features and the learning of channel-wise features. In much the same way that convolution relies on the assumption that the patterns in images are not tied to specific locations, depthwise separable convolution relies on the assumption that *spatial locations* in intermediate activations are *highly correlated*, but *different channels* are *highly independent*. Because this assumption is generally true for the image representations learned by deep

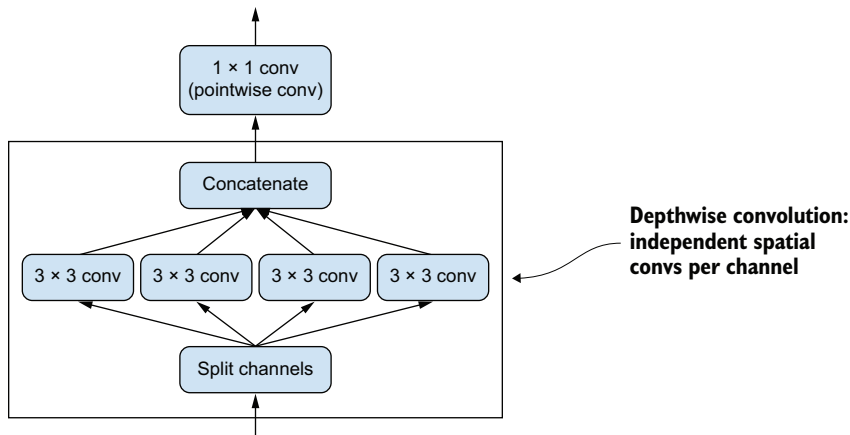


Figure 9.4 Depthwise separable convolution: a depthwise convolution followed by a pointwise convolution

neural networks, it serves as a useful prior that helps the model make more efficient use of its training data. A model with stronger priors about the structure of the information it will have to process is a better model—as long as the priors are accurate.

Depthwise separable convolution requires significantly fewer parameters and involves fewer computations compared to regular convolution, while having comparable representational power. They result in smaller models that converge faster and are less prone to overfitting. These advantages become especially important when you’re training small models from scratch on limited data.

When it comes to larger-scale models, depthwise separable convolutions are the basis of the Xception architecture, a high-performing ConvNet that comes packaged with Keras. You can read more about the theoretical grounding for depthwise separable convolutions and Xception in the paper “Xception: Deep Learning with Depthwise Separable Convolutions.”³

The co-evolution of hardware, software, and algorithms

Consider a regular convolution operation with a 3×3 window, 64 input channels, and 64 output channels. It uses $3 \times 3 \times 64 \times 64 = 36,864$ trainable parameters, and when you apply it to an image, it runs a number of floating-point operations that is proportional to this parameter count. Meanwhile, consider an equivalent depthwise separable convolution: it only involves $3 \times 3 \times 64 + 64 \times 64 = 4,672$ trainable parameters and proportionally fewer floating-point operations. This efficiency improvement only increases as the number of filters or the size of the convolution windows gets larger.

³ François Chollet, “Xception: Deep Learning with Depthwise Separable Convolutions,” Conference on Computer Vision and Pattern Recognition (2017), <https://arxiv.org/abs/1610.02357>.

(continued)

As a result, you would expect depthwise separable convolutions to be dramatically faster, right? Hold on. This would be true if you were writing simple CUDA or C++ implementations of these algorithms—in fact, you do see a meaningful speedup when running on CPU, where the underlying implementation is parallelized C++. But in practice, you’re probably using a GPU, and what you’re executing on it is far from a “simple” CUDA implementation: it’s a *cuDNN kernel*, a piece of code that has been extraordinarily optimized, down to each machine instruction. It certainly makes sense to spend a lot of effort optimizing this code, since cuDNN convolutions on NVIDIA hardware are responsible for many exaflops of computation every day. But a side effect of this extreme micro-optimization is that alternative approaches have little chance to compete on performance—even approaches that have significant intrinsic advantages, like depthwise separable convolutions.

Despite repeated requests to NVIDIA, depthwise separable convolutions have not benefited from nearly the same level of software and hardware optimization as regular convolutions, and as a result they remain only about as fast as regular convolutions, even though they’re using quadratically fewer parameters and floating-point operations. Note, though, that using depthwise separable convolutions remains a good idea even if it does not result in a speedup: their lower parameter count means that you are less at risk of overfitting, and their assumption that channels should be uncorrelated leads to faster model convergence and more robust representations.

What is a slight inconvenience in this case can become an impassable wall in other situations: because the entire hardware and software ecosystem of deep learning has been micro-optimized for a very specific set of algorithms (in particular, ConvNets trained via backpropagation), there’s an extremely high cost to steering away from the beaten path. If you were to experiment with alternative algorithms, such as gradient-free optimization or spiking neural networks, the first few parallel C++ or CUDA implementations you’d come up with would be orders of magnitude slower than a good old ConvNet—no matter how clever and efficient your ideas were. Convincing other researchers to adopt your method would be a tough sell, even if it were just plain better.

You could say that modern deep learning is the product of a co-evolution process between hardware, software, and algorithms: the availability of NVIDIA GPUs and CUDA led to the early success of backpropagation-trained ConvNets, which led NVIDIA to optimize its hardware and software for these algorithms, which in turn led to consolidation of the research community behind these methods. At this point, figuring out a different path would require a multiyear reengineering of the entire ecosystem.

9.5 *Putting it together: A mini Xception-like model*

As a reminder, here are the ConvNet architecture principles you’ve learned so far:

- Your model should be organized into repeated *blocks* of layers, usually made of multiple convolution layers and a max pooling layer.
- The number of filters in your layers should increase as the size of the spatial feature maps decreases.

- Deep and narrow is better than broad and shallow.
- Introducing residual connections around blocks of layers helps you train deeper networks.
- It can be beneficial to introduce batch normalization layers after your convolution layers.
- It can be beneficial to replace `Conv2D` layers with `SeparableConv2D` layers, which are more parameter efficient.

Let's bring all of these ideas together into a single model. Its architecture resembles a smaller version of Xception. We'll apply it to the dogs-versus-cats task from last chapter. For data loading and model training, simply reuse the exact same setup as what we used in chapter 8, section 8.2—but replace the model definition with the following `ConvNet`:

We apply a series of convolutional blocks with increasing feature depth. Each block consists of two batch-normalized depthwise separable convolution layers and a max pooling layer, with a residual connection around the entire block.

The assumption that underlies separable convolution, “Feature channels are largely independent,” does not hold for RGB images! Red, green, and blue color channels are actually highly correlated in natural images. As such, the first layer in our model is a regular `Conv2D` layer. We'll start using `SeparableConv2D` afterward.

```
import keras

inputs = keras.Input(shape=(180, 180, 3))
x = layers.Rescaling(1.0 / 255)(inputs)
x = layers.Conv2D(filters=32, kernel_size=5, use_bias=False)(x)

for size in [32, 64, 128, 256, 512]:
    residual = x

    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.SeparableConv2D(size, 3, padding="same", use_bias=False)(x)

    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.SeparableConv2D(size, 3, padding="same", use_bias=False)(x)

    x = layers.MaxPooling2D(3, strides=2, padding="same")(x)

    residual = layers.Conv2D(
        size, 1, strides=2, padding="same", use_bias=False
    )(residual)
    x = layers.add([x, residual])

x = layers.GlobalAveragePooling2D()(x)
```

Don't forget input rescaling!

In the original model, we used a Flatten layer before the Dense layer. Here, we go with a GlobalAveragePooling2D layer.

```
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs=inputs, outputs=outputs)
```

Like in the original model, we add a dropout layer for regularization.

This ConvNet has a trainable parameter count of 721,857, significantly lower than the 1,569,089 trainable parameters of the model from the previous chapter, yet it achieves better results. Figure 9.5 shows the training and validation curves.

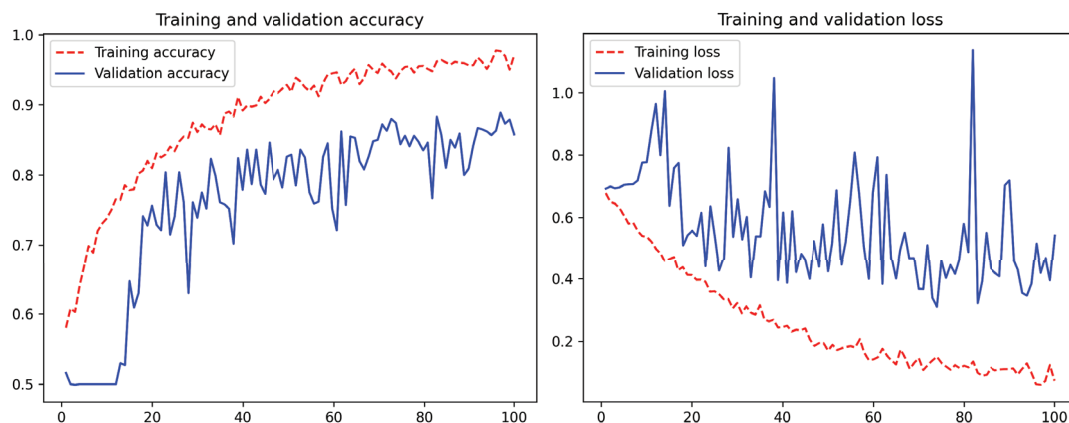


Figure 9.5 Training and validation metrics with a Xception-like architecture

You'll find that our new model achieves a test accuracy of 90.8%—compared to 83.9% for the previous model. As you can see, following architecture best practices does have an immediate, sizeable effect on model performance!

At this point, if you want to further improve performance, you should start systematically tuning the hyperparameters of your architecture—a topic we cover in detail in chapter 18. We haven't gone through this step here, so the configuration of the previous model is purely from the best practices we outlined, plus, when it comes to gauging model size, a small amount of intuition.

9.6 *Beyond convolution: Vision Transformers*

While ConvNets have been dominating the field of computer vision since the mid-2010s, they've been recently competing with an alternative architecture: Vision Transformers (or ViTs for short). It may well be that ViTs will end up replacing ConvNets in the long term—though, for now, ConvNets remain your best option in most cases.

You don't yet know what Transformers are because we'll cover them in chapter 15. In short, the Transformer architecture was developed to process text—it's fundamentally

a sequence-processing architecture. And Transformers are very good at it, which has led to the question: could we also use them for images?

Because ViTs are a type of Transformer, they also process sequences: they split up an image into a 1D sequence of patches, turn each patch into a flat vector, and process the vector sequence. The Transformer architecture allows ViTs to capture long-range relationships between different parts of the image, something ConvNets can sometimes struggle with.

Our general experience with Transformers is that they're a great choice if you're working with a massive dataset. They're simply better at utilizing large amounts of data. However, for smaller datasets, they tend to be suboptimal for two reasons. First, they lack the spatial prior of ConvNets—the 2D patch-based architecture of ConvNets incorporates more assumptions about the local structure of the visual space, making them more data efficient. Second, for ViTs to shine, they need to be really large. They end up being unwieldy for anything smaller than ImageNet.

The battle for image recognition supremacy is far from over, but ViTs have undoubtedly opened a new and exciting chapter. You'll probably work with this architecture in the context of large-scale generative image models—a topic we'll cover in chapter 17. For your small-scale image classification needs, however, ConvNets remain your best bet.

This concludes our introduction to essential ConvNet architecture best practices. With these principles in hand, you'll be able to develop higher-performing models across a wide range of computer vision tasks. You're now well on your way to becoming a proficient computer vision practitioner. To further deepen your expertise, there's one last important topic we need to cover: interpreting how a model arrives at its predictions.

Summary

- The architecture of a deep learning model encodes key assumptions about the nature of the problem at hand.
- The modularity-hierarchy-reuse formula underpins the architecture of nearly all complex systems, including deep learning models.
- Key architecture patterns for computer vision include residual connections, batch normalization, and depthwise separable convolutions.
- Vision Transformers are an up-and-coming alternative to ConvNets for large-scale computer vision tasks.